15 Apr 2024

Advancements in Sequence Generation: A GAN-Based Reinforcement Learning Approach

Advancements in Sequence Generation: A GAN-Based Reinforcement Learning Approach

Janet Rodriguez [1]

I. INTRODUCTION

This paper builds upon the SeqGAN framework, aiming to introduce targeted enhancements to improve its efficacy. SeqGAN, a pivotal algorithm for generating discrete token sequences, is extensively utilized across various NLP applications. By conceptualizing the data generator as a stochastic policy within the domain of reinforcement learning (RL), SeqGAN adeptly sidesteps the challenges associated with generator differentiation through direct gradient policy updates. The RL reward signal is derived from the evaluations of a GAN discriminator assessing complete sequences, subsequently informing state-action decisions via Monte Carlo search. Notable enhancements include the incorporation of WGAN, employing Earth Mover's distance rather than KL divergence for better generator guidance. Additionally, the adoption of proximal policy optimization is explored to further refine generator performance.

II. MODEL DESCRIPTION

A. Sequence Generative Adversarial Nets

The sequence generation problem is denoted as follows. Given a dataset of real-world structured sequences, train a θ-parameterized generative model G_θ to produce a sequence $Y_{1:T} = (y_1, \cdots, y_t, \cdots, y_T), y_t \in \mathbb{Y}$, where $\mathbb{Y}$ is the vocabulary of candidate tokens. We interpret this problem based on reinforcement learning. In timestep t, the state s is the current produced tokens $(y_1, \cdots, y_t, \cdots, y_{t-1})$ and the action a is the next token y_t to select. Thus the policy model $G_\theta(y_t|Y_{1:t-1})$ is stochastic, whereas the state transition is deterministic after an action has been chosen, i.e. $\delta^a_{s,s'} = 1$ for the next state $s' = Y_{1:t}$ it the current state $s = Y_{1:t-1}$and the action $a = y_t$; for other next states s'', $\delta^a_{s,s''} = 0$.

Additionally, we also train a ϕ-parameterized discriminative model D_ϕ (Goodfellow and others 2014) to provide a guidance for improving generator G_θ. $D_\phi(Y_{1:T})$ is a probability indicating how likely a sequence $Y_{1:T}$ is from real sequence data or not. As illustated in Figure 1, the discriminative model D_ϕ is trained by providing positive examples from the real sequence data and negative examples from the synthetic sequences generated from the generative model G_θ. At the same time, the generative model G_θ is updated by employing a policy gradient and MC search on the basis of the expected end reward received from the discriminative model D_ϕ. The reward is estimated by the likelihood that it would fool the discriminative model D_ϕ. The specific formulation is given in the next subsection.

Janet Rodriguez Email: dmurphy@gmail.com

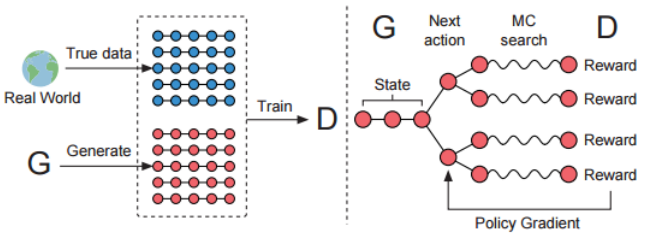

Fig. 1. Depicts the SeqGAN model. On the left: the discriminator D trains on both real and synthetic data produced by G. On the right: G undergoes training via policy gradients, receiving a final reward signal from D, which is relayed back through Monte Carlo search to intermediate actions.

B. SeqGAN via Policy Gradient

Following (Sutton et al. 1999), when there is no intermediate reward, the objective of the generator model (policy) $G_\theta(y_t|Y_{1:t-1})$ is to generate a sequence from the start state s_0 to maximize its expected end reward:

$$J(\theta) = E[R_T|s_0, \theta] = \sum_{y_1 \in \mathbb{Y}} G_\theta(y_1|s_0) \cdot Q^{G_\theta}_{D_\phi}(s_0, y_1) \quad (1)$$

where R_T is the reward for a complete sequence. Note that the reward is from the discriminator D_ϕ, which we will discuss later. $Q^{G_\theta}_{D_\phi}(s, a)$ is the action-value function of a sequence, i.e. the expected accumulative reward starting from state s, taking action a, and then following policy G_θ. The rational of the objective function for a sequence is that starting from a given initial state, the goal of the generator is to generate a sequence which would make the discriminator consider it is real.

The next question is how to estimate the action-value function. In this paper, we use the REINFORCE algorithm (Williams 1992) and consider the estimated probability of being real by the discriminator $D_\phi(Y^n_{1:T})$ as the reward. Formally, we have:

$$Q^{G_\theta}_{D_\phi}(a = y_T, s = Y_{1:T-1}) = D_\phi(Y_{1:T}). \quad (2)$$

However, the discriminator only provides a reward value for a finished sequence. Since we actually care about the longterm reward, at every timestep, we should not only consider the fitness of previous tokens (prefix) but also the resulted future outcome. This is similar to playing the games such as Go or Chess where players sometimes would give up the immediate interests for the long-term victory (Silver et al. 2016). Thus, to evaluate the action-value for an intermediate state, we apply Monte Carlo search with a roll-out policy

G_β to sample the unknown last $T-t$ tokens. We represent an $N-$time Monnte Carlo search as

$$\{Y_{1:T}^1, \cdots, Y_{1:T}^N\} = MC^{G_\beta}(Y_{1:t}; N). \quad (3)$$

where $Y_{1:t}^n = (y_1, \cdots, y_t)$ and $Y_{t+1:T}^n$ is sampled based on the roll-out policy G_β and the current state. In our experiment, G_β is set the same as the generator, but one can use a simplified version if the speed is the priority (Silver et al. 2016). To reduce the variance and get more accurate assessment of the action value, we run the roll-out policy starting from current state till the end of the sequence for N times to get a batch of output samples. Thus, we have:

$$Q_{D_\phi}^{G_\theta}(a = y_T, s = Y_{1:T-1}) = \quad (4)$$

$$\begin{cases} \frac{1}{N}\sum_{n=1}^{N} D_\phi(Y_{1:T}^n),\ Y_{1:T}^n \in MC^{G_\beta}(Y_{1:t}; N) & \text{for } t < T \\ D_\phi(Y_{1:T}) & \text{for } t = T \end{cases}$$

where, we see that when no intermediate reward, the function is iteratively defined as the next-state value starting from state $s' = Y_{1:T}$ and rolling out to the end.

A benefit of using the discriminator D_ϕ as a reward function is that it can be dynamically updated to further improve the generative model iteratively. Once we have a set of more realistic generated sequences, we shall re-train the discriminator model as follows:

$$\min_\phi -E_{Y\sim p_{data}}[logD_\phi(Y)] - E_{Y\sim G_\theta}[log(1-D_\phi(Y))] \quad (5)$$

Each time when a new discriminator model has been obtained, we are ready to update the generator. The proposed policy based method relies upon optimizing a parametrized policy to directly maximize the long-term reward. Following (Sutton et al. 1999), the gradient of the objective function $J(\theta)$ w.r.t. the generator's parameters θ can be derived as

$$\nabla_\theta J(\theta) = E_{Y_{1:t-1}\sim G_\theta}[\sum_{y_t\in\mathbb{Y}} \nabla_\theta G_\theta(y_t|Y_{1:t-1}) \cdot Q_{D_\phi}^{G_\theta}(Y_{1:t-1}, y_t)] \quad (6)$$

The above form is due to the deterministic state transition and zero intermediate rewards. Using likelihood ratios (Glynn 1990; Sutton et al. 1999), we build an unbiased estimation for Eq. (6) (on one episode):

$$\nabla_\theta J(\theta) \simeq \frac{1}{T}\sum_{t=1}^{T}\sum_{y_t\in\mathbb{Y}} \nabla_\theta G_\theta(y_t|Y_{1:t-1}) \cdot Q_{D_\phi}^{G_\theta}(Y_{1:t-1}, y_t)$$

$$= \frac{1}{T}\sum_{t=1}^{T}\sum_{y_t\in\mathbb{Y}} G_\theta(y_t|Y_{1:t-1}) \nabla_\theta log G_\theta(y_t|Y_{1:t-1}) \cdot Q_{D_\phi}^{G_\theta}(Y_{1:t-1}, y_t)$$

$$= \frac{1}{T}\sum_{t=1}^{T} E_{y_t\sim G_\theta(y_t|Y_{1:t-1})} \nabla_\theta log G_\theta(y_t|Y_{1:t-1}) \cdot Q_{D_\phi}^{G_\theta}(Y_{1:t-1}, y_t) \quad (7)$$

where $Y_{1:t-1}$ is the observed intermediate state sampled from G_θ. Since the expectation $E[\cdot]$ can be approximated by sampling methods, we then update the generator's parameters as:

$$\theta \leftarrow \theta + \alpha_h \nabla_\theta J(\theta)$$

where $\alpha_h \in \mathbb{R}^+$ denotes the corresponding learning rate at h-th step. Also the advanced gradient algorithms such as Adam and RMSprop can be adopted here.

In summary, Algorithm 1 shows full details of the proposed SeqGAN. At the beginning of the training, we use the maximum likelihood estimation (MLE) to pre-train G_θ on training set $\mathbb{S}$. We found the supervised signal from the pretrained discriminator is informative to help adjust the generator efficiently.

After the pre-training, the generator and discriminator are trained alternatively. As the generator gets progressed via training on g-steps updates, the discriminator needs to be retrained periodically to keeps a good pace with the generator. When training the discriminator, positive examples are from the given dataset S, whereas negative examples are generated from our generator. In order to keep the balance, the number of negative examples we generate for each d-step is the same as the positive examples. And to reduce the variability of the estimation, we use different sets of negative samples combined with positive ones, which is similar to bootstrapping (Quinlan 1996).

Algorithm 1 Euclid's algorithm

Require: generator policy G_θ; roll-out policy G_β; discriminator D_ϕ; a sequence dataset $\mathbb{S} = \{X_{1:T}\}$

Initialize G_θ, D_ϕ with random weights θ, ϕ.
Pre-train G_θ using MLE on $\mathbb{S}$
$\beta \leftarrow \theta$
Generate negative samples using G_θ for training D_ϕ
Pre-train D_ϕ via minimizing the cross entropy
repeat
  for $g-$step **do**
    Generate a sequence $Y_{1:T} = (y_1, \cdots, y_T) \sim G_\theta$
    for t in 1:T **do**
      Compute $Q(a = y_t; s = Y_{1:t-1})$ by Eq. (4)
    end for
    Update generator parameters via policy gradient Eq. (8)
  end for
  for d-step **do**
    Use current G_θ to generate negative examples and combine with given positive examples $\mathbb{S}$.
    Train discriminator D_ϕ for k epochs by Eq. (5)
  end for
  $\beta \leftarrow \theta$
until SeqGAN converges

C. The Generative Model for Sequences

We use recurrent neural networks (RNNs) (Hochreiter and Schmidhuber 1997) as the generative model. An RNN maps the input embedding representations $\boldsymbol{x_1}, \cdots, \boldsymbol{x_T}$ of

the sequence $x_1, \cdots, x_T$ into a sequence of hidden states $\boldsymbol{h_1}, \cdots, \boldsymbol{h_T}$ by using the update function g recursively.

$$\boldsymbol{h_t} = g(\boldsymbol{h_{t-1}}, \boldsymbol{x_t}) \tag{9}$$

Moreover, a softmax output layer z maps the hidden states into the output token distribution

$$p(y_t|x_1, \cdots, x_t) = z(\boldsymbol{h_t}) = softmax(\boldsymbol{c} + \boldsymbol{V}\boldsymbol{h_t}) \tag{10}$$

where the parameters are a bias vector c and a weight matrix $\boldsymbol{V}$. To deal with the common vanishing and exploding gradient problem (Goodfellow, Bengio, and Courville 2016) of the backpropagation through time, we leverage the Long Short-Term Memory (LSTM) cells (Hochreiter and Schmidhuber 1997) to implement the update function g in Eq. (9). It is worth noticing that most of the RNN variants, such as the gated recurrent unit (GRU) (Cho et al. 2014) and soft attention mechanism (Bahdanau, Cho, and Bengio 2014), can be used as a generator in SeqGAN.

D. The Discriminative Model for Sequences

Deep discriminative models such as deep neural network (DNN) (Vesel'y et al. 2013), convolutional neural network (CNN) (Kim 2014) and recurrent convolutional neural network (RCNN) (Lai et al. 2015) have shown a high performance in complicated sequence classification tasks. In this paper, we choose the CNN as our discriminator as CNN has recently been shown of great effectiveness in text (token sequence) classification (Zhang and LeCun 2015). Most discriminative models can only perform classification well for an entire sequence rather than the unfinished one. In this paper, we also focus on the situation where the discriminator predicts the probability that a finished sequence is real. In our work, the generated sequence has a fixed length T, but note that CNN is also capable of the variable-length sequence discrimination with the max-over-time pooling technique (Kim 2014).

We first represent an input sequence $x_1, \cdots, x_T$ as:

$$\varepsilon_{1:T} = \boldsymbol{x_1} \oplus \boldsymbol{x_2} \oplus \cdots \oplus \boldsymbol{x_T}, \tag{11}$$

where $\boldsymbol{x_t} \in \mathbb{R}^k$ is the $k-$dimensional token embedding and $\oplus$ is the concatenation operator to build the matrix $\varepsilon_{1:T} \in \mathbb{R}^{T\times k}$. Then a kernel $\boldsymbol{w} \in \mathbb{R}^{l\times k}$ applies a convolutional operation to a window size of l words to produce a new feature map:

$$c_i = \rho(\boldsymbol{w} \otimes \boldsymbol{\varepsilon_{i:i+l-1}} + b) \tag{12}$$

where $\otimes$ operator is the summation of elementwise production, b is a bias term and ρ is a non-linear function. We can use various numbers of kernels with different window sizes to extract different features. Finally we apply a max-over-time pooling operation over the feature maps $\tilde{c} = max\{c_1, \cdots, c_{T-l+1}\}$.

To enhance the performance, we also add the highway architecture (Srivastava, Greff, and Schmidhuber 2015) based on the pooled feature maps. Finally, a fully connected layer with sigmoid activation is used to output the probability that the input sequence is real. The optimization target is to minimize the cross entropy between the ground truth label and the predicted probability as formulated in Eq. (5).

Detailed implementations of the generative and discriminative models are provided later.

III. Possible Improvement

A. WGAN and Improved WGAN

Wasserstein GAN uses Wasserstein distance for the discriminator to measure the distance between the generated sequences and the real sequences. It shares a lot of good properties without too many changes on the original GAN.

- Remove the nonlinear sigmoid transformation for the last layer of the discriminator
- Don't use the logarithm for the loss of both discriminator and generator.
- Everytime after the update of the parameter of the discriminator, clip it within an interval $[-c, c]$
- Don't use the momentum based optimization algorithm, e.g. Adam. Recommend to use RMSProp or SGD.

Here we only changed the training of the discriminator and keep the reinforcement learning part for the generator. Therefore eq. (5) changes to

$$\underset{\|D_\phi\|\leq 1}{max}\, E_{Y\sim p_{data}}[D_\phi(Y)] - E_{Y\sim G_\theta}[D_\phi(Y)]$$

In a following paper[5], people proposed a gradient penalty term $(\|\nabla_x D_\phi(x)\| - 1)^2$ on the loss function to get an approximate 1-Lipschitz discriminator.

B. Proximal Policy Optimization

When implementing policy gradient methods, people often face the problem of tuning the learning rate. When it is very small, the algorithm will converge too slow, while if the learning rate is too large, the algorithm might diverge or collapse. Trust Region Polocy Optimization (TRPO)[9] is a method that could theoretically guarantee that the policy would become better after each step of updating. But it is relatively complicated. Then, people proposed a simpler method called proximal policy optimization (PPO)[10] to use the clipped surrogate objective below to do the optimization of the policy.

$$L^{CPI}(\theta) = \hat{E}_t[\frac{\pi_\theta(a_t|s_t)}{\pi_{old}(a_t|s_t)}\hat{A}_t] = \hat{E}_t[r_t(\theta)\hat{A}_t]$$

$$L^{CLIP}(\theta) = [min(r(\theta)\hat{A}_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)]$$

CPI refers to conservative policy iteration. Without a constraint, maximization of L^{CPI} would lead to an excessively large policy update; hence, people proposed the L^{CLIP} to penalize changes to the policy that move $r_t(\theta)$ away from 1.

C. Increase Batch size

Recently there is a paper called 'Don't decay the learning rate, increase the batch size'. So we could just try our experiment by increasing the batch size ten times and see the results.

IV. Synthetic Data Experiments

To test the efficacy and add our understanding of SeqGAN, we conduct a simulated test with synthetic data2. To simulate the real-world structured sequences, we consider a language model to capture the dependency of the tokens. We use a randomly initialized LSTM as the true model, aka, the oracle, to generate the real data distribution $p(x_t|x_1, \cdots, x_{t-1})$ for the following experiments.

A. Evaluation Metric

The benefit of having such oracle is that firstly, it provides the training dataset and secondly evaluates the exact performance of the generative models, which will not be possible with real data. We know that MLE is trying to minimize the cross-entropy between the true data distribution p and our approximation q, i.e. $-E_{x\sim p} log q(x)$. However, the most accurate way of evaluating generative models is that we draw some samples from it and let human observers review them based on their prior knowledge. We assume that the human observer has learned an accurate model of the natural distribution $p_{human}(x)$. Then in order to increase the chance of passing Turing Test, we actually need to minimize the exact opposite average negative log-likelihood $-E_{x\sim q} log_{human}(x)$, with the role of p and q exchanged. In our synthetic data experiments, we can consider the oracle to be the human observer for real-world problems, thus a perfect evaluation metric should be

$$NLL_{oracle} = -E_{Y_{1:T}\sim G_\theta}[\sum_{t=1}^{T} log G_{oracle}(y_t|Y_{1:t-1})]$$

where G_θ and G_{oracle} denote our generatife model and the oracle respectively.

At the test stage, we use G_θ to generate 100,000 sequence samples and calculate NLL_{oracle} for each sample by G_{oracle} and their average score. Also significance tests are performed to compare the statistical properties of the generation performance between the baselines and SeqGAN.

B. Training Setting

To set up the synthetic data experiments, we first initialize the parameters of an LSTM network following the normal distribution $\mathbb{N}(0; 1)$ as the oracle describing the real data distribution $G_oracle(x_t|x_1, \cdots, x_{t-1})$. Then we use it to generate 10,000 sequences of length 20 as the training set $\mathbb{S}$ for the generative models.

In SeqGAN algorithm, the training set for the discriminator is comprised by the generated examples with the label 0 and the instances from $\mathbb{S}$ with the label 1. For different tasks, one should design specific structure for the convolutional layer and in our synthetic data experiments, the kernel size is from 1 to T and the number of each kernel size is between 100 to 200[3]. Dropout (Srivastava et al. 2014) and L2 regularization are used to avoid over-fitting.

Four generative models are compared with SeqGAN. The first model is a random token generation. The second one is the MLE trained LSTM G_θ. The third one is scheduled sampling (Bengio et al. 2015). The fourth one is the Policy Gradient with BLEU (PG-BLEU). In the scheduled sampling, the training process gradually changes from a fully guided scheme feeding the true previous tokens into LSTM, towards a less guided scheme which mostly feeds the LSTM with its generated tokens. A curriculum rate ! is used to control the probability of replacing the true tokens with the generated ones. To get a good and stable performance, we decrease ! by 0.002 for every training epoch. In the PG-BLEU algorithm, we use BLEU, a metric measuring the similarity between a generated sequence and references (training data), to score the finished samples from Monte Carlo search.

V. Results

TABLE I

Sequence generation performance comparison

Algorithm	Random	MLE	SS	PG-BLEU	SeqGAN
NLL	10.310	9.038	8.985	8.946	**8.639**

The NLL_{oracle} performance of generating sequences from the compared policies is provided in Table 1. Since the evaluation metric is fundamentally instructive, we can see the impact of SeqGAN, which outperforms other baselines significantly.

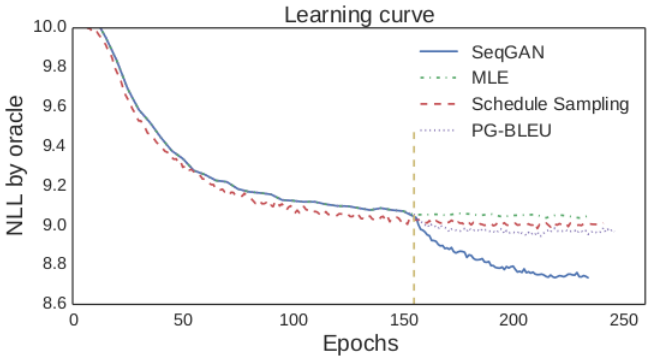

Fig. 2. Negative log-likelihood convergence w.r.t. the training epochs. The vertical dashed line represents the end of pre-training for SeqGAN, SS and PG-BLEU.

The learning curves shown in Figure 2 illustrate the superiority of SeqGAN explicitly. After about 150 training epochs, both the maximum likelihood estimation and the schedule sampling methods converge to a relatively high NLLoracle score, whereas SeqGAN can improve the limit of the generator with the same structure as the baselines significantly. This indicates the prospect of applying adversarial training strategies to discrete sequence generative models to breakthrough the limitations of MLE. Additionally, SeqGAN outperforms PG-BLEU, which means the discriminative signal in GAN is more general and effective than a predefined score (e.g. BLEU) to guide the generative policy to capture the underlying distribution of the sequence data.

TABLE II
COMPARISION OF SEQGAN WITH THE IMPOSSIBLE IMPROVEMENT

Algorithm	SeqGAN	WGAN	I-WGAN	I-Batch	PPO	l-PPO
NLL	8.639	8.824	**8.509**	8.567	9.065	8.729

The NLL_{oracle} performance of generating sequences from the possible improved models are provided in Table 2. We can see that the Improve WGAN could provide a better result, which means a good evaluation of the policy is the key to improve the performance of the generator. We could also show that the batch increasing method is valid for generating discrete tokens.

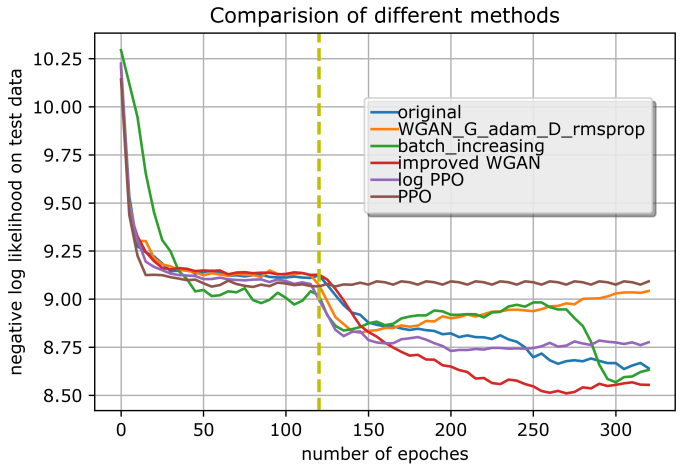

Fig. 3. Negative log-likelihood convergence w.r.t. the training epochs. The vertical dashed line represents the end of pre-training for SeqGAN, WGAN, Improved WGAN and batch increased SeqGAN.

The learning curves shown in Figure 3 illustrate the traing process of the various algorithms. We can see that after the pre-training phase, the generator guided by the discriminator could decrease the NLL_{oracle} sharply. But for WGAN, it converges fast to a local optimum and then the performance is getting worse, which is a phenomenon of overfitting. Maybe more fine tuning of the hyperparameters is needed. As to batch increasing method, it performs very similar as WGAN in the early stage, while the NLL_{oracle} decreases rapidly after about 150 epochs of adversarial training. That might because the algorithm struggles in a valley of local optimum at the beginning but finally the larger batch could provide the right direction to jump out of the valley. For improved WGAN, we can see that it is the most stable one to converge to the best result. But there is also a little bit overfitting. It could be solved by fine tuning the learning rate. For PPO, it's kind of surprising that it's performance is inferior to REINFORCE and it doesn't show any improvement at the adversarial stage. I tried to replace

$$r(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{old}(a_t|s_t)}$$

by

$$\frac{log\pi_\theta(a_t|s_t)}{log\pi_{old}(a_t|s_t)}$$

and change

$$L^{CLIP}(\theta) = [min(r(\theta)\hat{A}_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)]$$

to

$$L^{log}(\theta) = clip((\frac{log\pi_\theta(a_t|s_t)}{log\pi_{old}(a_t|s_t)}, 1-\epsilon, 1+\epsilon)\hat{A}_t)$$

to get the log-PPO. Although log-PPO is more stable and could get a better result, it still cannot beat SeqGAN. This might be an interesting topic that can be further studied in the future.

VI. FUTURE WORK

More work should be done on the policy part. Also we could try to set up a network to evaluate the partially generated sequence. In addition, we should balance the capacity of the generator and discriminator to make sure that the generator could sufficiently learn from the reward signal from the discriminator.

REFERENCES

[1] Lantao Yu, Weinan Zhang, Jun Wang, and Yong Yu. 2016a. SeqGAN: sequence generative adversarial nets with policy gradient. arXiv preprint arXiv:1609.05473.
[2] Kim, Y. 2014. Convolutional neural networks for sentence classification. arXiv:1408.5882.
[3] Papineni, K.; Roukos, S.; Ward, T.; and Zhu, W.-J. 2002. Bleu: a method for automatic evaluation of machine translation. In ACL, 311–318.
[4] M. Arjovsky, S. Chintala, and L. Bottou. Wasserstein GAN. arXiv preprint arXiv:1701.07875, 2017.
[5] Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron Courville. Improved training of wasserstein gans. arXiv preprint arXiv:1704.00028, 2017.
[6] Srivastava, R. K.; Greff, K.; and Schmidhuber, J. 2015. Highway networks. arXiv:1505.00387.
[7] C. Jin, R. Ge, P. Netrapalli, S. M. Kakade, and M. I. Jordan. How to escape saddle points efficiently. arXiv preprint arXiv:1703.00887, 2017.
[8] Chi Jin and Michael Jordan. How to Escape Saddle Points Efficiently, July 19, 2017. http://www.offconvex.org/2017/07/19/saddle-efficiency/
[9] Schulman, John, Levine, Sergey, Moritz, Philipp, Jordan, Michael I, and Abbeel, Pieter. Trust region policy optimization. arXiv preprint arXiv:1502.05477, 2015b.
[10] Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347.
[11] S. L. Smith, P.-J. Kindermans, and Q. V. Le. Don't decay the learning rate, increase the batch size. arXiv preprint arXiv:1711.00489, 2017.
[12] Goodfellow, I., et al. 2014. Generative adversarial nets. In NIPS, 2672–2680.
[13] Sutton, R. S.; McAllester, D. A.; Singh, S. P.; Mansour, Y.; et al. 1999. Policy gradient methods for reinforcement learning with function approximation. In NIPS, 1057–1063.
[14] Williams, R. J. 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine learning 8(3-4):229–256.
[15] Silver, D.; Huang, A.; Maddison, C. J.; Guez, A.; Sifre, L.; et al. 2016. Mastering the game of go with deep neural networks and tree search. Nature 529(7587):484–489.
[16] Sitao Luan, Mingde Zhao, Xiao-Wen Chang, Doina Precup (2019). Break the ceiling: Stronger multi-scale deep graph convolutional networks. Advances in neural information processing systems, 32.

[17] Chenqing Hua, Sitao Luan, Minkai Xu, Zhitao Ying, Jie Fu, Stefano Ermon, & Doina Precup (2023, November). MUDiff: Unified Diffusion for Complete Molecule Generation. In The Second Learning on Graphs Conference.

[18] Sitao Luan, Mingde Zhao, Chenqing Hua, Xiao-Wen Chang, Doina Precup (2020). Complete the missing half: Augmenting aggregation filtering with diversification for graph convolutional networks. In NeurIPS 2022 Workshop: New Frontiers in Graph Learning.

[19] Sitao Luan, Chenqing Hua, Qincheng Lu, Jiaqi Zhu, Mingde Zhao, Shuyuan Zhang, Xiao-Wen Chang, Doina Precup (2021). Is Heterophily A Real Nightmare For Graph Neural Networks To Do Node Classification?. arXiv preprint arXiv:2109.05641.

[20] Sitao Luan, Chenqing Hua, Qincheng Lu, Jiaqi Zhu, Mingde Zhao, Shuyuan Zhang, Xiao-Wen Chang, Doina Precup (2022). Revisiting heterophily for graph neural networks. Advances in neural information processing systems, 35, 1362-1375.

[21] Sitao Luan, Chenqing Hua, Minkai Xu, Qincheng Lu, Jiaqi Zhu, Xiao-Wen Chang, Jie Fu, Jure Leskovec, Doina Precup (2024). When Do Graph Neural Networks Help with Node Classification? Investigating the Homophily Principle on Node Distinguishability. Advances in Neural Information Processing Systems, 36.

[22] Sitao Luan, Chenqing Hua, Qincheng Lu, Jiaqi Zhu, Xiao-Wen Chang, Doina Precup (2023). When do we need graph neural networks for node classification?. In International Conference on Complex Networks and Their Applications (pp. 37-48). Cham: Springer Nature Switzerland.

[23] Sitao Luan, Mingde Zhao, Xiao-Wen Chang, Doina Precup (2023, November). Training matters: Unlocking potentials of deeper graph convolutional neural networks. In International Conference on Complex Networks and Their Applications (pp. 49-60). Cham: Springer Nature Switzerland.

[24] Mingde Zhao, Zhen Liu, Sitao Luan, Shuyuan Zhang, Doina Precup, Yoshua Bengio (2021). A consciousness-inspired planning agent for model-based reinforcement learning. Advances in neural information processing systems, 34, 1569-1581.